Verilog code for 16-bit single cycle MIPS processor

In this project, a 16-bit single-cycle MIPS processor is implemented in Verilog HDL. MIPS is an RISC processor, which is widely used by many universities in academic courses related to computer organization and architecture.

The Instruction Format and Instruction Set Architecture for the 16-bit single-cycle MIPS are as follows:

fpga4student.com

Name	Fields					Comments
Field size	3 bits	3 bits	3 bits	3 bits	4 bits	All MIPS-L instructions 16 bits
R-format	op	rs	rt	rd	funct	Arithmetic instruction format
I-format	op	rs	rt	Address/immediate		Transfer, branch, immediate format
J-format	op	target address				Jump instruction format

fpga4student.com

Name	Format	Example					Comments
		3 bits	3 bits	3 bits	3 bits	4 bits	
add	R	0	2	3	1	0	add $1,$2,$3
sub	R	0	2	3	1	1	sub $1,$2,$3
and	R	0	2	3	1	2	and $1,$2,$3
or	R	0	2	3	1	3	or $1,$2,$3
slt	R	0	2	3	1	4	slt $1,$2,$3
jr	R	0	7	0	0	8	jr $7
lw	I	4	2	1	7		lw $1, 7 ($2)
sw	I	5	2	1	7		sw $1, 7 ($2)
beq	I	6	1	2	7		beq $1,$2, 7
addi	I	7	2	1	7		addi $1,$2, 7
j	J	2	500				j 1000
jal	J	3	500				jal 1000
slti	I	1	2	1	7		slti $1,$2, 7

Below is the description for instructions being implemented in Verilog:

1. **Add : R[rd] = R[rs] + R[rt]**
2. **Subtract : R[rd] = R[rs] - R[rt]**
3. **And: R[rd] = R[rs] & R[rt]**
4. **Or : R[rd] = R[rs] | R[rt]**
5. **SLT: R[rd] = 1 if R[rs] < R[rt] else 0**
6. **Jr: PC=R[rs]**
7. **Lw: R[rt] = M[R[rs]+SignExtImm]**
8. **Sw : M[R[rs]+SignExtImm] = R[rt]**
9. **Beq : if(R[rs]==R[rt]) PC=PC+1+BranchAddr**
10. **Addi: R[rt] = R[rs] + SignExtImm**
11. **J : PC=JumpAddr**
12. **Jal : R[7]=PC+2;PC=JumpAddr**
13. **SLTI: R[rt] = 1 if R[rs] < imm else 0**

SignExtImm = { 9{immediate[6]}, imm

JumpAddr = { (PC+1)[15:13], address}

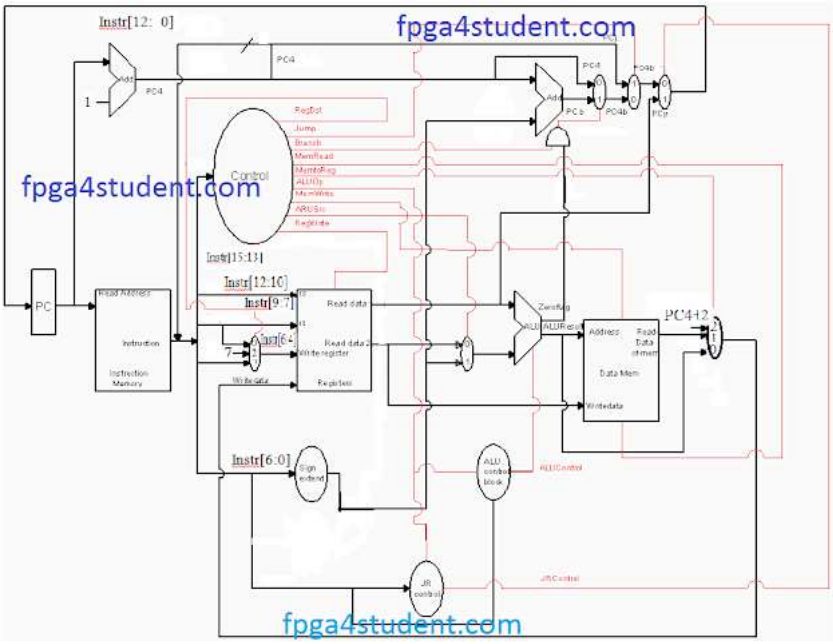
BranchAddr = { 7{immediate[6]}, immediate, 1'b0 }

Based on the provided instruction set, the data-path and control unit are designed and implemented.

Control unit design:

Control signals									
Instruction	Reg Dst	ALU Src	Memto Reg	Reg Write	MemRead	Mem Write	Branch	ALUOp	Jump
R-type	1	0	0	1	0	0	0	00	0
LW	0	1	1	1	1	0	0	11	0
SW	0	1	0	0	0	1	0	11	0
addi	0	1	0	1	0	0	0	11	0
beq	0	0	0	0	0	0	1	01	0
j	0	0	0	0	0	0	0	00	1
jal	2	0	2	1	0	0	0	00	1
slti	0	1	0	1	0	0	0	10	0

ALU Control				
ALU op	Function	ALUcnt	ALU Operation	Instruction
11	xxxx	000	ADD	Addi,lw,sw
01	xxxx	001	SUB	BEQ
00	00	000	ADD	R-type: ADD
00	01	001	SUB	R-type: sub
00	02	010	AND	R-type: AND
00	03	011	OR	R-type: OR
00	04	100	slt	R-type: slt
10	xxxxxx	100	slt	i-type: slti



After completing the design for the MIPS processor, it is easy to write Verilog code for the MIPS processor. The Verilog code for the whole design of the MIPS processor as follows:

Verilog code for ALU unit

Verilog code for register file

Verilog code for instruction memory

Verilog code for data memory:

```
//fpga4student.com: FPGA projects, Verilog projects, VHDL projects
// Verilog project: Verilog code for 16-bit MIPS Processor
// Submodule: Data memory in Verilog
 module data_memory
 (
      input                          clk,
      // address input, shared by read and write port
      input     [15:0]               mem_access_addr,
      // write port
      input     [15:0]               mem_write_data,
      input                          mem_write_en,
      input mem_read,
      // read port
      output     [15:0]               mem_read_data
 );
      integer i;
      reg [15:0] ram [255:0];
      wire [7 : 0] ram_addr = mem_access_addr[8 : 1];
      initial begin
           for(i=0;i<256;i=i+1)
                ram[i] <= 16'd0;
      end
      always @(posedge clk) begin
           if (mem_write_en)
                ram[ram_addr] <= mem_write_data;
      end
      assign mem_read_data = (mem_read==1'b1) ? ram[ram_addr]: 16'd0;
 endmodule
```

Verilog code for ALU Control unit:

```
//fpga4student.com: FPGA projects, Verilog projects, VHDL projects
// Verilog project: Verilog code for 16-bit MIPS Processor
// Submodule: ALU Control Unit in Verilog
 module ALUControl( ALU_Control, ALUOp, Function);
 output reg[2:0] ALU_Control;
 input [1:0] ALUOp;
 input [3:0] Function;
 wire [5:0] ALUControlIn;
 assign ALUControlIn = {ALUOp,Function};
 always @(ALUControlIn)
 casex (ALUControlIn)
  6'b11xxxx: ALU_Control=3'b000;
  6'b10xxxx: ALU_Control=3'b100;
  6'b01xxxx: ALU_Control=3'b001;
  6'b000000: ALU_Control=3'b000;
  6'b000001: ALU_Control=3'b001;
  6'b000010: ALU_Control=3'b010;
  6'b000011: ALU_Control=3'b011;
  6'b000100: ALU_Control=3'b100;
  default: ALU_Control=3'b000;
  endcase
 endmodule
// Verilog code for JR control unit
module JR_Control( input[1:0] alu_op,
       input [3:0] funct,
```

```
        output JRControl
    );
assign JRControl = ({alu_op,funct}==6'b001000) ? 1'b1 : 1'b0;
endmodule
```

Verilog code for control unit:

```
//fpga4student.com: FPGA projects, Verilog projects, VHDL projects
// Verilog project: Verilog code for 16-bit MIPS Processor
// Submodule: Control Unit in Verilog
 module control( input[2:0] opcode,
                           input reset,
                           output reg[1:0] reg_dst,mem_to_reg,alu_op,
                           output reg jump,branch,mem_read,mem_write,alu_src,reg_w
   );
 always @(*)
 begin
      if(reset == 1'b1) begin
                reg_dst = 2'b00;
                mem_to_reg = 2'b00;
                alu_op = 2'b00;
                jump = 1'b0;
                branch = 1'b0;
                mem_read = 1'b0;
                mem_write = 1'b0;
                alu_src = 1'b0;
                reg_write = 1'b0;
                sign_or_zero = 1'b1;
      end
      else begin
      case(opcode)
      3'b000: begin // add
                reg_dst = 2'b01;
                mem_to_reg = 2'b00;
                alu_op = 2'b00;
                jump = 1'b0;
                branch = 1'b0;
                mem_read = 1'b0;
                mem_write = 1'b0;
                alu_src = 1'b0;
                reg_write = 1'b1;
                sign_or_zero = 1'b1;
                end
      3'b001: begin // sli
                reg_dst = 2'b00;
                mem_to_reg = 2'b00;
                alu_op = 2'b10;
                jump = 1'b0;
                branch = 1'b0;
                mem_read = 1'b0;
                mem_write = 1'b0;
                alu_src = 1'b1;
                reg_write = 1'b1;
                sign_or_zero = 1'b0;
                end
      3'b010: begin // j
                reg_dst = 2'b00;
                mem_to_reg = 2'b00;
                alu_op = 2'b00;
                jump = 1'b1;
                branch = 1'b0;
                mem_read = 1'b0;
                mem_write = 1'b0;
                alu_src = 1'b0;
```

```
                reg_write = 1'b0;
                sign_or_zero = 1'b1;
                end
        3'b011: begin // jal
                reg_dst = 2'b10;
                mem_to_reg = 2'b10;
                alu_op = 2'b00;
                jump = 1'b1;
                branch = 1'b0;
                mem_read = 1'b0;
                mem_write = 1'b0;
                alu_src = 1'b0;
                reg_write = 1'b1;
                sign_or_zero = 1'b1;
                end
        3'b100: begin // lw
                reg_dst = 2'b00;
                mem_to_reg = 2'b01;
                alu_op = 2'b11;
                jump = 1'b0;
                branch = 1'b0;
                mem_read = 1'b1;
                mem_write = 1'b0;
                alu_src = 1'b1;
                reg_write = 1'b1;
                sign_or_zero = 1'b1;
                end
        3'b101: begin // sw
                reg_dst = 2'b00;
                mem_to_reg = 2'b00;
                alu_op = 2'b11;
                jump = 1'b0;
                branch = 1'b0;
                mem_read = 1'b0;
                mem_write = 1'b1;
                alu_src = 1'b1;
                reg_write = 1'b0;
                sign_or_zero = 1'b1;
                end
        3'b110: begin // beq
                reg_dst = 2'b00;
                mem_to_reg = 2'b00;
                alu_op = 2'b01;
                jump = 1'b0;
                branch = 1'b1;
                mem_read = 1'b0;
                mem_write = 1'b0;
                alu_src = 1'b0;
                reg_write = 1'b0;
                sign_or_zero = 1'b1;
                end
        3'b111: begin // addi
                reg_dst = 2'b00;
                mem_to_reg = 2'b00;
                alu_op = 2'b11;
                jump = 1'b0;
                branch = 1'b0;
                mem_read = 1'b0;
                mem_write = 1'b0;
                alu_src = 1'b1;
                reg_write = 1'b1;
                sign_or_zero = 1'b1;
                end
```

```
      default: begin
                reg_dst = 2'b01;
                mem_to_reg = 2'b00;
                alu_op = 2'b00;
                jump = 1'b0;
                branch = 1'b0;
                mem_read = 1'b0;
                mem_write = 1'b0;
                alu_src = 1'b0;
                reg_write = 1'b1;
                sign_or_zero = 1'b1;
                end
      endcase
      end
 end
 endmodule
```

Verilog code for the single-cycle MIPS processor:

```
//fpga4student.com: FPGA projects, Verilog projects, VHDL projects
// Verilog project: Verilog code for 16-bit MIPS Processor
 // Verilog code for 16 bit single cycle MIPS CPU
 module mips_16( input clk,reset,
                          output[15:0] pc_out, alu_result
                          //,reg3,reg4
   );
 reg[15:0] pc_current;
 wire signed[15:0] pc_next,pc2;
 wire [15:0] instr;
 wire[1:0] reg_dst,mem_to_reg,alu_op;
 wire jump,branch,mem_read,mem_write,alu_src,reg_write     ;
 wire     [2:0]     reg_write_dest;
 wire     [15:0] reg_write_data;
 wire     [2:0]     reg_read_addr_1;
 wire     [15:0] reg_read_data_1;
 wire     [2:0]     reg_read_addr_2;
 wire     [15:0] reg_read_data_2;
 wire [15:0] sign_ext_im,read_data2,zero_ext_im,imm_ext;
 wire JRControl;
 wire [2:0] ALU_Control;
 wire [15:0] ALU_out;
 wire zero_flag;
 wire signed[15:0] im_shift_1, PC_j, PC_beq, PC_4beq,PC_4beqj,PC_jr;
 wire beq_control;
 wire [14:0] jump_shift_1;
 wire [15:0]mem_read_data;
 wire [15:0] no_sign_ext;
 wire sign_or_zero;
 // PC
 always @(posedge clk or posedge reset)
 begin
      if(reset)
           pc_current <= 16'd0;
      else
           pc_current <= pc_next;
 end
 // PC + 2
 assign pc2 = pc_current + 16'd2;
 // instruction memory
 instr_mem instrucion_memory(.pc(pc_current),.instruction(instr));
 // jump shift left 1
 assign jump_shift_1 = {instr[13:0],1'b0};
 // control unit
 control control_unit(.reset(reset),.opcode(instr[15:13]),.reg_dst(reg_dst)
```

```
                ,.mem_to_reg(mem_to_reg),.alu_op(alu_op),.jump(jump),.branch(branc
                .mem_write(mem_write),.alu_src(alu_src),.reg_write(reg_write),.sig
 // multiplexer regdest
 assign reg_write_dest = (reg_dst==2'b10) ? 3'b111: ((reg_dst==2'b01) ? instr[6:4]
 // register file
 assign reg_read_addr_1 = instr[12:10];
 assign reg_read_addr_2 = instr[9:7];
 register_file reg_file(.clk(clk),.rst(reset),.reg_write_en(reg_write),
 .reg_write_dest(reg_write_dest),
 .reg_write_data(reg_write_data),
 .reg_read_addr_1(reg_read_addr_1),
 .reg_read_data_1(reg_read_data_1),
 .reg_read_addr_2(reg_read_addr_2),
 .reg_read_data_2(reg_read_data_2));
 //.reg3(reg3),
 //.reg4(reg4));
 // sign extend
 assign sign_ext_im = {{9{instr[6]}},instr[6:0]};
 assign zero_ext_im = {{9{1'b0}},instr[6:0]};
 assign imm_ext = (sign_or_zero==1'b1) ? sign_ext_im : zero_ext_im;
 // JR control
 JR_Control JRControl_unit(.alu_op(alu_op),.funct(instr[3:0]),.JRControl(JRControl
 // ALU control unit
 ALUControl ALU_Control_unit(.ALUOp(alu_op),.Function(instr[3:0]),.ALU_Control(ALU
 // multiplexer alu_src
 assign read_data2 = (alu_src==1'b1) ? imm_ext : reg_read_data_2;
 // ALU
 alu alu_unit(.a(reg_read_data_1),.b(read_data2),.alu_control(ALU_Control),.result
 // immediate shift 1
 assign im_shift_1 = {imm_ext[14:0],1'b0};
 //
 assign no_sign_ext = ~(im_shift_1) + 1'b1;
 // PC beq add
 assign PC_beq = (im_shift_1[15] == 1'b1) ? (pc2 - no_sign_ext): (pc2 +im_shift_1)
 // beq control
 assign beq_control = branch & zero_flag;
 // PC_beq
 assign PC_4beq = (beq_control==1'b1) ? PC_beq : pc2;
 // PC_j
 assign PC_j = {pc2[15],jump_shift_1};
 // PC_4beqj
 assign PC_4beqj = (jump == 1'b1) ? PC_j : PC_4beq;
 // PC_jr
 assign PC_jr = reg_read_data_1;
 // PC_next
 assign pc_next = (JRControl==1'b1) ? PC_jr : PC_4beqj;
 // data memory
 data_memory datamem(.clk(clk),.mem_access_addr(ALU_out),
 .mem_write_data(reg_read_data_2),.mem_write_en(mem_write),.mem_read(mem_read),
 .mem_read_data(mem_read_data));
 // write back
 assign reg_write_data = (mem_to_reg == 2'b10) ? pc2:((mem_to_reg == 2'b01)? mem_r
 // output
 assign pc_out = pc_current;
 assign alu_result = ALU_out;
 endmodule
```

Verilog testbench code for the single-cycle MIPS processor:

```
 `timescale 1ns / 1ps
//fpga4student.com: FPGA projects, Verilog projects, VHDL projects
// Verilog project: Verilog code for 16-bit MIPS Processor
// Testbench Verilog code for 16 bit single cycle MIPS CPU
 module tb_mips16;
```

```
        // Inputs
        reg clk;
        reg reset;
        // Outputs
        wire [15:0] pc_out;
        wire [15:0] alu_result;//,reg3,reg4;
        // Instantiate the Unit Under Test (UUT)
        mips_16 uut (
             .clk(clk),
             .reset(reset),
             .pc_out(pc_out),
             .alu_result(alu_result)
             //.reg3(reg3),
            // .reg4(reg4)
        );
        initial begin
             clk = 0;
             forever #10 clk = ~clk;
        end
        initial begin
             // Initialize Inputs
             //$monitor ("register 3=%d, register 4=%d", reg3,reg4);
             reset = 1;
             // Wait 100 ns for global reset to finish
             #100;
       reset = 0;
             // Add stimulus here
        end
  endmodule
```

It is quite simple to verify the Verilog code for the single-cycle MIPS CPU by doing several simulations on ModelSim or Xilinx ISIM in order to see how the MIPS processor works. To fully verify the MIPS processor, it is needed to modify the instruction memory to simulate all the instructions in the instruction set architecture, and then check simulation waveform and memory to see if the processor works correctly as designed.

Source code provided by
Abdrazak Mohammed Anana

You may like this:
32-bit 5-stage Pipelined MIPS Processor in Verilog (Part-1)
32-bit 5-stage Pipelined MIPS Processor in Verilog (Part-2)
32-bit 5-stage Pipelined MIPS Processor in Verilog (Part-3)
Verilog code for 16-bit RISC Processor
VHDL code for MIPS Processor
Recommended Verilog projects:

1. **What is an FPGA? How Verilog works on FPGA**
2. Verilog code for FIFO memory
3. Verilog code for 16-bit single-cycle MIPS processor
4. Programmable Digital Delay Timer in Verilog HDL
5. Verilog code for basic logic components in digital circuits
6. Verilog code for 32-bit Unsigned Divider
7. Verilog code for Fixed-Point Matrix Multiplication
8. Plate License Recognition in Verilog HDL
9. Verilog code for Carry-Look-Ahead Multiplier
10. Verilog code for a Microcontroller
11. Verilog code for 4x4 Multiplier
12. Verilog code for Car Parking System
13. Image processing on FPGA using Verilog HDL
14. How to load a text file into FPGA using Verilog HDL
15. Verilog code for Traffic Light Controller
16. Verilog code for Alarm Clock on FPGA

17. Verilog code for comparator design
18. Verilog code for D Flip Flop
19. Verilog code for Full Adder
20. Verilog code for counter with testbench
21. Verilog code for 16-bit RISC Processor
22. Verilog code for button debouncing on FPGA
23. How to write Verilog Testbench for bidirectional/ inout ports
24. Tic Tac Toe Game in Verilog and LogiSim
25. 32-bit 5-stage Pipelined MIPS Processor in Verilog (Part-1)
26. 32-bit 5-stage Pipelined MIPS Processor in Verilog (Part-2)
27. 32-bit 5-stage Pipelined MIPS Processor in Verilog (Part-3)
28. Verilog code for Decoder
29. Verilog code for Multiplexers
30. N-bit Adder Design in Verilog
31. Verilog vs VHDL: Explain by Examples
32. Verilog code for Clock divider on FPGA
33. How to generate a clock enable signal in Verilog
34. Verilog code for PWM Generator
35. Verilog coding vs Software Programming
36. Verilog code for Moore FSM Sequence Detector
37. Verilog code for 7-segment display controller on Basys 3 FPGA

DMCA PROTECTED

Privacy Policy | Disclaimer | Sitemap | Contact | Advertise Here
Copyright © 2016-2020 FPGA4student.com All Rights Reserved.